

Enterprise Self-Service Analytics & ML Platform

Project Specification & Build Brief — for AI-assisted development (Antigravity)

1. Project Overview

Build a self-service analytics and machine learning platform that allows business users to upload a dataset (CSV), automatically clean and explore the data, select a target column, and train an AutoML-style model without writing code. Results (metrics, feature importance, predictions) are displayed through an interactive dashboard. The system is containerized and deployable to a cloud environment (Azure or AWS) with an automated CI/CD pipeline.

2. Goals & Success Criteria

- User can upload a CSV dataset through the UI and receive an automated EDA summary within seconds.
- User can select a target column and trigger model training without writing any code.
- System automatically selects between classification and regression, tries multiple algorithms, and returns the best-performing model.
- Results view displays accuracy/performance metrics, feature importance, and sample predictions via interactive charts.
- Entire application runs via a single `docker-compose up` locally, and is deployed to the cloud with CI/CD triggering on every push to main.
- No unhandled exceptions on common edge cases: empty files, missing values, single-class targets, non-numeric target columns.

3. Tech Stack

Layer	Technology	Purpose
Backend API	Python, FastAPI	REST endpoints, data processing orchestration
ML Layer	scikit-learn, XGBoost	AutoML-style training, evaluation
Data Handling	pandas, numpy	Cleaning, EDA, transformations
Frontend	React or Streamlit	Upload UI, dashboards, charts
Visualization	Plotly / Chart.js	Feature importance, metrics, prediction charts
Containerization	Docker, Docker Compose	Local + cloud deployment consistency
Cloud	Azure (App Service/AKS) or AWS (ECS/EKS)	Hosting, scalability
Storage	Azure Blob / AWS S3 (or local disk for MVP)	Dataset and model artifact storage
CI/CD	GitHub Actions	Automated test, build, deploy on push

4. High-Level Architecture

Client (React/Streamlit) → FastAPI Backend → [Data Module | ML Module | Storage Module] → Response (JSON) → Client renders charts. All services run in separate Docker containers, orchestrated via Docker Compose locally and

deployed as independent services in the cloud (backend as an API service, frontend as a static/SSR app, storage as managed cloud storage).

5. Module-Wise Breakdown

Module 1 — Data Ingestion & Validation

- Endpoint: POST /upload — accepts CSV file, validates type/size (max 50MB for MVP).
- Store file temporarily (local disk for MVP, S3/Blob for cloud).
- Return a dataset_id used in all subsequent requests.
- Reject malformed files with clear error messages (not silent failure).

Module 2 — EDA & Cleaning

- Endpoint: GET /eda/{dataset_id} — returns column types, null counts, basic stats, sample rows.
- Endpoint: POST /clean/{dataset_id} — handles missing values (impute/drop), encodes categoricals, flags outliers.
- Cleaning options should be configurable via request parameters, not hardcoded.
- All transformations must be logged so results are explainable to the user.

Module 3 — Model Training (AutoML)

- Endpoint: POST /train — accepts dataset_id and target_column.
- Auto-detect classification vs regression based on target column dtype/cardinality.
- Train 2-3 candidate models (e.g., RandomForest, XGBoost, LogisticRegression/LinearRegression) using cross-validation.
- Select best model by relevant metric (F1/accuracy for classification, RMSE/R2 for regression).
- Persist trained model + metadata (joblib) tied to a job_id.
- Handle edge cases explicitly: single-class target, all-null target, non-numeric target with no valid encoding path.

Module 4 — Results & Explainability

- Endpoint: GET /results/{job_id} — returns metrics, feature importance, and sample predictions as JSON.
- Feature importance computed via model-native methods (e.g., .feature_importances_) or SHAP if time allows.
- Response schema must be fixed and documented (use Pydantic models) so frontend never receives unexpected shapes.

Module 5 — Frontend Dashboard

- Upload screen: drag-and-drop CSV upload, shows EDA summary immediately after upload.
- Target column selector: dropdown populated dynamically from uploaded dataset's columns.
- Training trigger: initiates training, shows loading/progress state (polling or WebSocket).
- Results view: metrics table + feature importance bar chart + prediction sample table, using Plotly/Chart.js.
- Graceful error states for failed uploads, failed training, and invalid target columns.

Module 6 — Containerization

- Separate Dockerfiles for backend and frontend, each with minimal final image size.

- docker-compose.yml orchestrating backend, frontend, and storage/DB (if used) with environment variables via .env.
- Health-check endpoints (/health) on backend for container orchestration readiness checks.

Module 7 — Cloud Deployment

- Push Docker images to a container registry (ACR for Azure, ECR for AWS).
- Deploy backend to Azure App Service/AKS or AWS ECS/EC2 first; verify via public health-check URL.
- Deploy frontend pointing to the live backend URL.
- Configure cloud storage (Blob/S3) for datasets and model artifacts in place of local disk.
- Set environment-specific configs via secrets/environment variables — never hardcoded credentials.

Module 8 — CI/CD Pipeline

- GitHub Actions workflow triggered on push to main.
- Pipeline steps: lint → run tests (pytest) → build Docker image → push to registry → trigger cloud redeploy.
- Keep initial pipeline minimal; expand with staging environments once core flow is stable.

6. Non-Functional Requirements

- All API responses must follow strict Pydantic schemas — no ad hoc JSON shapes.
- Every module should be independently testable via direct API calls before frontend integration.
- Error handling must return meaningful HTTP status codes and messages, not generic 500s.
- Secrets (API keys, DB credentials) must be managed via environment variables, never committed to source.
- Target: full local stack running via a single docker-compose command; full cloud deployment reachable via public URL.

7. Deliverables

- Working codebase (backend + frontend) in a Git repository with clear folder structure.
- Dockerfiles + docker-compose.yml for local run.
- Deployed, publicly accessible instance (Azure or AWS).
- GitHub Actions CI/CD pipeline configuration.
- README with architecture diagram, setup instructions, and API documentation.

Prepared as a build brief for AI-assisted development. Modules are ordered by dependency — each should be built and tested in isolation before integration with the next.